The mutex and condition variable are now contained within the `threadsafe_queue` instance, so separate variables are no longer required ❶, and no external synchronization is required for the call to `push()` ❷. Also, `wait_and_pop()` takes care of the condition variable wait ❸.

The other overload of `wait_and_pop()` is now trivial to write, and the remaining functions can be copied almost verbatim from the stack example in listing 3.5. The final queue implementation is shown here.

Listing 4.5. Full class definition for a thread-safe queue using condition variables

```
#include <queue>
#include <memory>
#include <mutex>
#include <condition_variable>

template<typename T>
class threadsafe_queue
{
private:
    mutable std::mutex mut;
    std::queue<T> data_queue;
    std::condition_variable data_cond;
public:
    threadsafe_queue()
    {}
    threadsafe_queue(threadsafe_queue const& other)
    {
        std::lock_guard<std::mutex> lk(other.mut);
        data_queue=other.data_queue;

    }

    void push(T new_value)
    {
        std::lock_guard<std::mutex> lk(mut);
        data_queue.push(new_value);
        data_cond.notify_one();
    }

    void wait_and_pop(T& value)
    {
        std::unique_lock<std::mutex> lk(mut);
        data_cond.wait(lk,[this]{return !data_queue.empty();});
        value=data_queue.front();
        data_queue.pop();
    }
```

❶ The mutex must be mutable

Even though `empty()` is a `const` member function, and the `other` parameter to the copy constructor is a `const` reference, other threads may have non-`const` references to the object, and be calling mutating member functions, so we still need to lock the mutex. Since locking a mutex is a mutating operation, the mutex object must be marked `mutable` ❶ so it can be locked in `empty()` and in the copy constructor.

Condition variables are also useful where there's more than one thread waiting for the same event. If the threads are being used to divide the workload, and thus only one thread should respond to a notification, exactly the same structure as shown in listing 4.1 can be used; just run multiple instances of the data—processing thread. When new data is ready, the call to `notify_one()` will trigger one of the threads currently executing `wait()` to check its condition and thus return from